SOURCE https://www.underworldcode.org/boundary-conditions-on-non-planar-boundaries/
UNDERWORLD underworld3 development @ 0addec15
LICENCE CC-BY-4.0
VERSION 1.0.0
ARTICLE UWTN 2026-016
PUBLISHED Aug 18, 2026

Boundary conditions on non-planar boundaries

Louis Moresi [1]

[1] Australian National University

Keywords Underworld Code, Tricks of the Trade, development

A free surface moves under the traction it carries. That makes the wall-normal stress the quantity driving the model rather than something read off at the end, and it is the reason we went back to how the boundary condition underneath it is imposed.

The condition itself is ordinary. Free slip is no flow through the boundary and no tangential drag along it,

$$\boldsymbol{u} \cdot \hat{\boldsymbol{n}} = 0 \qquad \text{and} \qquad \hat{\boldsymbol{t}} \cdot \boldsymbol{\sigma} \hat{\boldsymbol{n}} = 0. \tag{1}$$

On a box the first of those is a single velocity component. You hold u_x on a vertical wall, the solver removes a row, and there is nothing further to discuss. On a sphere, an annulus, a mesh that has been moved, or a surface with topography, $\boldsymbol{u} \cdot \hat{\boldsymbol{n}}$ is not a component of anything. That is the whole difficulty, and everything below is a way around it.

The second condition is worth naming because it is the one people forget. Zero tangential traction is *natural*: it is what you get by leaving the boundary term out of the weak form. Nothing has to be done to impose it, and something has to be done to avoid imposing it accidentally.

1. Where the boundary term comes from

Every method below is a statement about one term. Multiplying the momentum balance by a test function $\boldsymbol{w}$ and integrating by parts gives

$$\int_\Omega \boldsymbol{\sigma} : \nabla \boldsymbol{w} \, \mathrm{d}V - \int_{\partial\Omega} (\boldsymbol{\sigma}\hat{\boldsymbol{n}}) \cdot \boldsymbol{w} \, \mathrm{d}S = \int_\Omega \boldsymbol{f} \cdot \boldsymbol{w} \, \mathrm{d}V. \tag{2}$$

Drop the surface integral and you have imposed zero traction in both directions — free *everything*, not free slip. Free slip keeps the tangential half of that and replaces the normal half with the constraint. How you do the replacing is the choice.

2. Four ways, in order

2.a. A direct penalty

Add a term that punishes any flow through the boundary:

$$\ldots + \frac{\gamma}{h} \int_{\partial\Omega} (\boldsymbol{u}\cdot\hat{\boldsymbol{n}})(\boldsymbol{w}\cdot\hat{\boldsymbol{n}})\, \mathrm{d}S. \tag{3}$$

One line, no new machinery, and it works on any geometry. It is still in a good many working scripts, and deservedly. What you are solving is a perturbed problem, though, and it is perturbed by exactly the amount the constraint is violated: the discrete solution sits where the penalty term balances the traction it is fighting, which leaves $\boldsymbol{u}\cdot\hat{\boldsymbol{n}}$ small but not zero. Making it smaller means pushing harder, and pushing harder conditions the operator worse. The error is traded against the conditioning, and measured below, that trade runs out: past 10^4 the leak stops improving, and by 10^6 the solve fails. Nitsche moves the floor down rather than removing it — it fails too, at its own threshold.

Underworld spells it as a boundary traction opposing normal flow, using the facet normal at the quadrature points:

```
G = mesh.Gamma
stokes.add_natural_bc(1.0e4 * G.dot(v.sym) * G, "Upper")
```

2.b. Nitsche

The reason the penalty is only accurate in the limit is that it is not *consistent*: substituting the true solution does not leave the equation satisfied, because the true solution is subject to a boundary traction the penalty form ignores. Nitsche's method (Nitsche, 1971) restores consistency by carrying that traction explicitly:

$$\ldots - \int_{\partial\Omega} (\hat{\boldsymbol{n}}\cdot\boldsymbol{\sigma}(\boldsymbol{u})\hat{\boldsymbol{n}})(\boldsymbol{w}\cdot\hat{\boldsymbol{n}})\, \mathrm{d}S - \int_{\partial\Omega} (\hat{\boldsymbol{n}}\cdot\boldsymbol{\sigma}(\boldsymbol{w})\hat{\boldsymbol{n}})(\boldsymbol{u}\cdot\hat{\boldsymbol{n}})\, \mathrm{d}S + \frac{\gamma}{h} \int_{\partial\Omega} (\boldsymbol{u}\cdot\hat{\boldsymbol{n}})(\boldsymbol{w}\cdot\hat{\boldsymbol{n}})\, \mathrm{d}S. \tag{4}$$

The first of the three is the consistency term: it is the boundary traction the integration by parts produced, and putting it back is what makes the true solution satisfy the discrete equations exactly. The second is its transpose, which keeps the form symmetric and buys optimal convergence in L^2. The third is the penalty again, and it is still needed — but now for *stability* rather than for accuracy, and γ has a threshold set by an inverse inequality rather than being a dial you turn up until the answer looks right.

This is a real improvement and it is still a weak imposition. The constraint holds to the accuracy of the discretisation, not to the accuracy of the arithmetic — measured below, it leaks a few parts in a thousand on a mesh of the resolution people actually run, and the leak falls with the mesh rather than with the machine.

2.c. A constraint equation, with a multiplier

The two above add a *term* to an equation. This one adds an *equation*.

Carry a scalar field h on the boundary and require, as a row of the system in its own right,

$$\int_{\partial\Omega} (\boldsymbol{u}\cdot\hat{\boldsymbol{n}} - g)\, q\, \mathrm{d}S = 0 \quad \text{for all } q, \tag{5}$$

with h entering the momentum row as the traction $h\hat{\boldsymbol{n}}$ that holds the constraint. It is a Lagrange multiplier, and the system becomes a larger saddle point: velocity, pressure, and now h.

Nothing is being traded here. The constraint row is exact, so unlike a penalty there is no parameter whose size decides how well it holds. Two practical things do have to be dealt with, and they are where the approximation enters:

- The multiplier is only defined on the boundary, so its interior degrees of freedom are singular. Underworld screens them with a small ε (default 10^{-6}), which is the one place the constraint is relaxed.
- The $[p, h]$ Schur complement is poorly conditioned on its own, so an augmented-Lagrangian term $r(\boldsymbol{u} \cdot \hat{\boldsymbol{n}} - g)\hat{\boldsymbol{n}}$ is added to the momentum row. This conditions the block **without biasing the multiplier**, because the h row still carries the exact constraint — so unlike a penalty, the accuracy does not depend on r.

```
stokes = uw.systems.Stokes_Constrained(mesh, velocityField=v, pressureField=p)
h = stokes.add_constraint_bc(0.0, "Upper")
stokes.solve()
```

And the reason to care about it beyond the constraint: **at convergence, h on the boundary is the normal traction.** It is not recovered from the velocity field afterwards — it is an unknown the solve returned, available through `multiplier` and, divided by $\Delta\rho g$, through `topography`.

2.d. Rotating the degrees of freedom

Stop asking for the constraint and impose it. At each constrained node, change the basis in which the velocity unknowns are expressed, from the global Cartesian frame to the local $(\hat{\boldsymbol{n}}, \hat{\boldsymbol{t}})$ frame. In that basis "no flow through the boundary" is again a single component, and it is removed the same way it would be on a box.

Collect the per-node rotations into a block-diagonal Q, equal to the identity at every node that is not constrained. The rotated system is

$$\hat{A} = Q^T A Q, \qquad \hat{\boldsymbol{b}} = Q^T \boldsymbol{b}, \qquad \boldsymbol{u} = Q\hat{\boldsymbol{u}}, \tag{6}$$

and the wall-normal row of $\hat{A}$ is struck out. The constraint then holds to machine precision, because it is not being solved for at all.

This is the classical answer, not a new one. It is in the early finite-element literature, and Engelman et al. (1982) were already reviewing the alternatives and choosing between them on grounds of global mass conservation in 1982. What is worth explaining is not the idea but why, given that it is exact and the others are not, it is the least used of the three.

3. What it costs, and the cost is structural

Rotating the degrees of freedom leaves the discrete problem in a **mixed basis**. Interior nodes hold (u_x, u_y); constrained nodes hold (u_n, u_t). Nothing about that is difficult in itself, and everything downstream has to agree about which nodes are which.

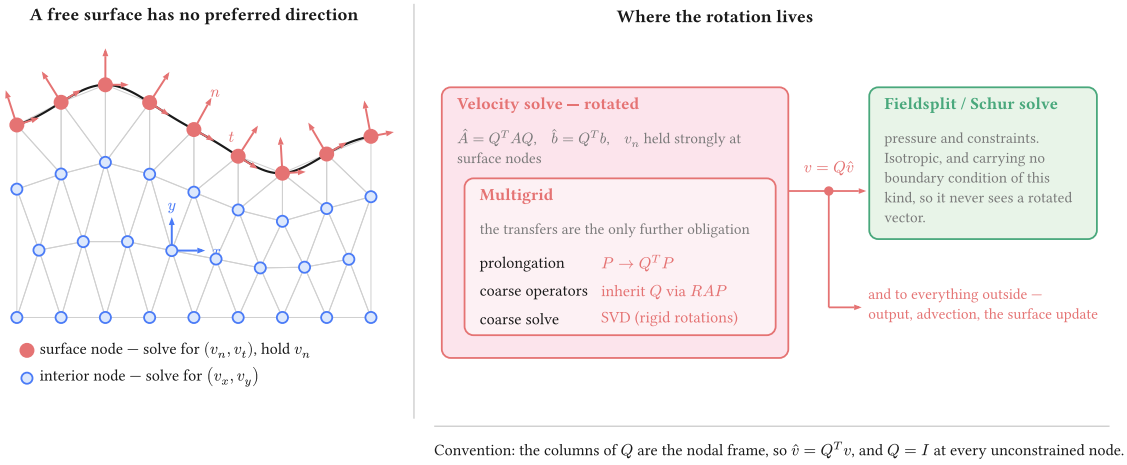

Figure 1: Where the rotation lives. The obligation is contained: the velocity solve is rotated and carries its multigrid with it, while the Schur complement and the pressure solve beside it never handle a rotated vector, because the pressure block carries no boundary condition of this kind. One un-rotation sits on the boundary between them and feeds both.

Four things carry Q: the operator, the right-hand side, the solution on the way out, and the multigrid prolongation. The coarse operators inherit it through the Galerkin triple product rather than being rotated separately, and the coarse solve then has to be an SVD, because a Galerkin-coarsened rotated operator inherits the rigid-rotation null space of the constrained problem and a redundant LU factorisation meets a zero pivot in it.

That last point is worth dwelling on, because it is the one that surprises. The constraint never has to be re-imposed on a coarse mesh — which is fortunate, since we install no discretisation there and could not impose it if we wanted to. It arrives anyway, algebraically, through $P^T \hat{A} P$ with a rotated P. The evidence that it really arrives is that the coarse operator inherits the null space, which is a property only the constrained problem has.

None of this is an argument against the method. It is an argument for knowing what is being taken on, and it is the honest reason a weakly imposed condition survives in codes that could do this instead.

4. Which normal

A question with a less obvious answer than it looks. The boundary of a discretised domain is a set of straight facets, and the assembled constraint is an integral over those facets. The node normal consistent with that integral is the average of the adjacent facet normals **weighted by facet measure** — not the normal of the smooth surface the mesh approximates.

Using the analytic normal is exact for the geometry and therefore inconsistent with the discretisation, which is the wrong way round: the solver is not solving on the sphere, it is solving on the polyhedron. In parallel the same argument has a sharper edge, because a normal accumulated rank-locally is wrong at a partition boundary, where a node's facets are split across ranks and no rank sees them all.

Worth checking against the literature

Behr (2004) addresses this question directly for slip conditions on curved boundaries. We have not read it against our own derivation. If it reaches the same measure-weighted normal, this section should say so and cite it as the source rather than presenting the result as ours.

5. The option this note leaves out

Solving in spherical or cylindrical components makes the wall-normal direction a coordinate direction again, and the constraint returns to being "hold one component". That is the same rotation as above, applied once for the whole domain instead of node by node, and applied that way it costs nothing structurally: there is no mixed basis, because every node is in the same basis.

It works exactly when the boundary lies along a coordinate surface. A sphere, an annulus, a cylinder. It does nothing for topography, for a mesh that has been deformed, or for a tilted internal surface, which is the general case this note is about. The per-node rotation is what remains once the geometry stops cooperating, and its structural price is what buys the generality.

6. When the choice matters

Here is the awkward part. Solve a convection model with any of the three and the velocity field is the same to plotting accuracy. A leak of order 10^{-3} in $\boldsymbol{u} \cdot \hat{\boldsymbol{n}}$ is invisible to anything that consumes the velocity, and consuming the velocity is most of what a model does. If that is your situation, use the simplest thing that works and stop reading here.

The difference appears when the wall-normal traction is the answer rather than a by-product: dynamic topography, a plate-boundary force balance, anything compared against a geoid or a gravity field. Then the three separate, and they separate for a structural reason rather than by a matter of degree.

Under a penalty or a Nitsche condition the constraint is approximate, so a traction recovered from the solution inherits the approximation — you are differentiating a field that was never made to satisfy the condition exactly. Under the rotated constraint the reaction *is* σ_{nn}: it is the multiplier the solve has already computed, and it comes out of `boundary_normal_traction` without a recovery step or an augmented-Lagrangian splitting.

6.a. The leak, measured

An annulus, no slip on the inner radius, the treatment under test on the outer, driven by a degree-four radial density anomaly. The number is the largest normal velocity on the outer boundary, taken against the true radial direction and divided by the flow speed: the fraction of the flow going through a boundary nothing should pass through. Penalty at 10^4, Nitsche at $\gamma = 10$.

cell size	direct penalty	Nitsche	multiplier	rotated
0.150	4.5×10^{-3}	4.6×10^{-3}	8.3×10^{-4}	7.3×10^{-11}
0.100	2.5×10^{-3}	2.2×10^{-3}	1.6×10^{-4}	6.5×10^{-11}
0.075	8.5×10^{-4}	1.7×10^{-3}	5.6×10^{-5}	1.0×10^{-10}
0.050	9.5×10^{-4}	5.8×10^{-4}	1.2×10^{-5}	8.4×10^{-11}

The refinement is what separates them, and a single resolution would not have. The two weak forms leak parts in a thousand and improve roughly as $h^{1.9}$ — the rate consistency buys. The multiplier starts an order of magnitude better and falls much faster, near $h^{3.9}$, because its only approximation is the screening of the interior multiplier degrees of freedom rather than the enforcement itself. The rotated constraint does not move at all: it sits at the solver's floor at every resolution, because the mesh has nothing to do with it.

The control matters more than the result. With the outer boundary left free the same measurement reads **0.98** — nearly all the boundary flow is normal — so the metric can see a leak when there is one.

6.b. What each parameter buys

The two weak methods look alike in that table. They are not alike, and their own parameters are what tells them apart.

penalty	leak		γ (Nitsche)	leak
10^2	2.6×10^{-1}		1	diverged
10^3	2.3×10^{-2}		10	1.7×10^{-3}
10^4	8.5×10^{-4}		100	2.7×10^{-4}
10^5	9.6×10^{-4}		1000	3.0×10^{-5}
10^6	diverged		10^4 and above	diverged

Both have a wall, and neither escapes tuning. The direct penalty improves in proportion to how hard it pushes until the conditioning catches up: no gain from 10^4 to 10^5, and failure at 10^6. Nitsche improves further and then fails outright — at $\gamma = 10^4$ the line search stops converging, and the figures above that are from solves that did not finish.

What differs is how far down each gets before it fails. The penalty bottoms out near 10^{-3}; Nitsche reaches 3×10^{-5}, better by more than an order of magnitude. That is what consistency buys — a smaller floor, not an unbounded parameter.

Nitsche also fails *below* $\gamma = 1$, and for the opposite reason: the form is no longer coercive, and no amount of tuning recovers it. So its usable range is bounded at both ends, with the stability threshold underneath and the conditioning above. The virtue of $\gamma = 10$ is that it sits in the middle of that window on any mesh, because γ is dimensionless and the term it scales already carries μ/h. The penalty coefficient carries no such scaling, which is why the value that works is a property of the problem rather than a default.

> **What is measured here, and what is not**
>
> This measures the *constraint*. The claim that follows — that a traction recovered from a constraint good to 10^{-3} inherits that error, while the rotated reaction is exact because it is the multiplier itself — follows from how each is computed, and the surface-stress comparison against a known answer has not been run. Until it is, the two paragraphs above this section are reasoning rather than measurement.

7. Using it

```python
import underworld3 as uw

mesh = uw.meshing.Annulus(radiusInner=0.5, radiusOuter=1.0, cellSize=0.05)
stokes = uw.systems.Stokes(mesh)

# Value first: 0 is free slip. A non-zero scalar or expression prescribes the
# wall-normal datum u.n = u_n strongly instead.
stokes.add_rotated_freeslip_bc(0.0, "Upper")
stokes.add_rotated_freeslip_bc(0.0, "Lower")
```

```
stokes.solve()

# The constraint reaction, which is the boundary normal traction.
sigma_nn = stokes.boundary_normal_traction("Upper")
```

Leave the normal to Underworld unless the constraint has to follow the true surface rather than the mesh. Passing an analytic normal — `X / |X|` on a sphere — is exact for the geometry and keeps a consistency error against the faceted assembly, which is usually not what you want.

Reach for Nitsche when the boundary condition has to **change during the model**. A hard constraint cannot morph: a wall that begins as a prescribed velocity and relaxes to a prescribed traction is a Nitsche problem, because the rotated constraint is either imposed or it is not.

References

Behr, M. (2004). On the application of slip boundary condition on curved boundaries. *International Journal for Numerical Methods in Fluids*, *45*(1), 43–51. https://doi.org/10.1002/fld.663

Engelman, M. S., Sani, R. L., & Gresho, P. M. (1982). The implementation of normal and/or tangential boundary conditions in finite element codes for incompressible fluid flow. *International Journal for Numerical Methods in Fluids*, *2*(3), 225–238. https://doi.org/10.1002/fld.1650020302

Nitsche, J. (1971). ̈Uber ein Variationsprinzip zur Lösung von Dirichlet-Problemen bei Verwendung von Teilräumen, die keinen Randbedingungen unterworfen sind. *Abhandlungen Aus Dem Mathematischen Seminar Der Universität Hamburg*, *36*(1), 9–15. https://doi.org/10.1007/bf02995904